

Politechnika Wrocławska

Sztuczna Inteligencja i Inżynieria Wiedzy

Raport 4

**Klasyfikacja przeżywalności pacjentów z marskością wątroby
metodami uczenia maszynowego**

Dawid Pilarski

Numer indeksu: 280468

Wrocław, 2026

1. Wprowadzenie

Tematem tej listy jest praktyczne zastosowanie metod uczenia maszynowego do rozwiązania medycznego problemu klasyfikacji. Celem jest przejście całej ścieżki projektu opartego o ML: od eksploracji danych i zdefiniowania problemu, przez przygotowanie danych, dobór algorytmów i ich hiperparametrów, aż po ocenę modeli i interpretację wyników.

W przeciwieństwie do poprzedniej listy, gdzie opisywaliśmy świat, a rozwiązanie generował gotowy planer, tutaj sami budujemy modele, które uczą się na danych historycznych i na ich podstawie podejmują decyzje o nowych, niewidzianych wcześniej przypadkach.

Pracuję na zbiorze „Cirrhosis Patient Survival Prediction” z repozytorium UCI. Realizuję dwa wymagane klasyfikatory - naiwny klasyfikator Bayesa oraz drzewo decyzyjne - a w ramach zadań bonusowych testuję algorytmy zaawansowane (Random Forest, SVM) oraz badam jedno podejście do łągodzenia przeuczenia. Implementację oparłem o Pythona i bibliotekę scikit-learn, a wyniki zebrałem w notebooku Jupyter.

2. Opis problemu

Marskość wątroby jest wynikiem długotrwałego uszkodzenia narządu, prowadzącego do rozległego bliznowacenia. Najczęstsze przyczyny to przewlekłe zapalenie wątroby oraz nadużywanie alkoholu. Zbiór danych pochodzi z badania Mayo Clinic prowadzonego w latach 1974-1984 i liczy 418 przypadków.

Każdy pacjent opisany jest 17 cechami, które dzielą się na trzy grupy:

- **dane demograficzne** - wiek (w dniach) i płeć,
- **objawy fizyczne** - obecność wodobrzusza (Ascites), powiększenia wątroby (Hepatomegaly), naczynek gwiaździstych (Spiders), obrzęków (Edema) oraz histologiczne stadium choroby (Stage),
- **wyniki badań biochemicznych** - m.in. poziom bilirubiny, cholesterolu, albuminy, miedzi, fosfatazy alkalicznej (Alk_Phos), SGOT, trójglicerydów, liczba płytek krwi (Platelets) i czas protrombinowy (Prothrombin).

Dodatkowo zbiór zawiera informację o podawanym leku (Drug: D-penicylamina lub placebo).

Zmienną docelową jest atrybut **Status**, przyporządkowujący pacjentowi jeden z trzech możliwych wyników na koniec okresu obserwacji:

- **D** (Death) - zgon w wyniku choroby,
- **C** (Censored) - wynik oceniany, tzn. pacjent żył na koniec badania lub zmarł z przyczyny niezwiązanej z chorobą,
- **CL** (Censored due to Liver transplant) - wynik oceniany z powodu przeszczepu wątroby.

Problem jest więc **klasyfikacją wieloklasową** (3 klasy). Zadaniem modelu jest, na podstawie 17 cech klinicznych, przypisać pacjenta do jednej z tych grup.

Dwie cechy tego problemu mają kluczowe znaczenie dla całego dalszego postępowania i wracają w każdej sekcji raportu:

Niezbalansowanie klas. Klasy nie są równoliczne - klasa CL (przeszczep) jest bardzo rzadka i stanowi tylko niewielki procent zbioru. To znaczy, że dokładność (accuracy) jako jedyna metryka jest myląca: model całkowicie ignorujący najrzadszą klasę i tak osiągnie wysoki wynik. Z tego powodu jako podstawowe kryterium oceny i strojenia przyjmuję **F1-macro** (które traktuje wszystkie klasy równo), a dokładność raportuję pomocniczo.

Wyciek informacji a kolumna N_Days. Surowy zbiór zawiera kolumnę N_Days - liczbę dni od rejestracji do zgonu, przeszczepu lub końca badania. Ta kolumna nie znajduje się na liście 17 cech zadania i celowo ją usuwam. Powód: N_Days jest bezpośrednio powiązana z wynikiem (dla pacjenta zmarłego liczba dni urywa się w momencie zgonu), więc pozostawienie jej w danych byłoby **wyciekiem targetu (data leakage)** - model „przewidywałby” coś, co w praktyce już zna. Sztucznie zawyżyłoby to wszystkie metryki i uczyniło ocenę bezwartościową. Analogicznie usuwam kolumnę ID jako czysty identyfikator bez wartości predykcyjnej.

3. Tło teoretyczne

Poniżej krótko opisuję pojęcia potrzebne w dalszej części raportu. Rozwiązany problem to klasyfikacja, czyli uczenie nadzorowane, w którym model na podstawie cech przewiduje etykietę będącą kategorią - tutaj jedną z trzech klas Status.

3.1. Użyte klasyfikatory

Naiwny klasyfikator Bayesa (GaussianNB) to model probabilistyczny oparty na twierdzeniu Bayesa, który naiwnie zakłada wzajemną niezależność cech, a dla cech liczbowych przyjmuje ich normalny rozkład w obrębie klasy. Drzewo decyzyjne dzieli dane serią pytań, wybierając w każdym węźle podział najlepiej rozdzielający klasy (miarą jest entropia i przyrost informacji). Jego główną wadą jest przeuczenie: przy dużej głębokości uczy się danych treningowych na pamięć i traci zdolność do generalizacji, co widać po dużej różnicy między wynikiem na treningu a na walidacji. Ogranicza się je między innymi głębokością drzewa lub przycinaniem.

3.2. Metody przetwarzania danych

Standaryzacja i normalizacja ujednolicają skalę cech, ale nie zmieniają wyników drzewa ani gaussowskiego Bayesa, dlatego do porównania używam metod, które realnie na nie wpływają: selekcji cech (SelectKBest z informacją wzajemną, która odpowiada przyrostowi informacji) oraz PCA, czyli redukcji wymiaru przez rzutowanie danych na składowe główne wyjaśniające najwięcej wariancji.

3.3. Metryki oceny

Ocenę opieram na macierzy pomyłek i wyprowadzonych z niej miarach: dokładności, precyzji, czułości oraz F1 (średniej harmonicznej precyzji i czułości). Przy niezbalansowanych klasach ważne jest rozróżnienie uśrednień: macro-F1 liczy F1 osobno dla każdej klasy i uśrednia bez wag (rzadka klasa CL waży tyle samo co liczna C), a weighted uśrednia proporcjonalnie do liczności klas. Duża różnica między nimi sygnalizuje, że model dobrze radzi sobie z klasami licznymi, a słabo z rzadką.

4. Eksploracja danych

Eksplorację wykonałem na zbiorze po wczytaniu, ale przed jakimkolwiek przetwarzaniem, żeby zobaczyć dane „takie, jakie są”.

Rozkład klasy docelowej. Najważniejsza jest tu proporcja klas:

	liczność	udział_%
C	232	55.5
CL	25	6.0
D	161	38.5

Zbiór jest mocno niezbalansowany. Klasa C to ponad połowa przypadków (232 z 418), D około 38%, a CL zaledwie 6%, czyli 25 pacjentów - z czego w zbiorze walidacyjnym zostaje ich tylko około pięciu. Przy takiej dysproporcji sama dokładność niewiele mówi, bo model może w ogóle nie rozpoznawać klasy CL i mimo to osiągać wysoki wynik, trafiając w dwie liczne klasy. Dlatego jako główną metrykę przyjmuję macro-F1, które liczy każdą klasę osobno i traktuje je równo, a dokładność podaję tylko pomocniczo.

Statystyki cech liczbowych.

	mean	std	min	25%	50%	75%	max
Age	18533.35	3815.85	9598.00	15644.50	18628.00	21272.50	28650.00
Bilirubin	3.22	4.41	0.30	0.80	1.40	3.40	28.00
Cholesterol	369.51	231.94	120.00	249.50	309.50	400.00	1775.00
Albumin	3.50	0.42	1.96	3.24	3.53	3.77	4.64
Copper	97.65	85.61	4.00	41.25	73.00	123.00	588.00
Alk_Phos	1982.66	2140.39	289.00	871.50	1259.00	1980.00	13862.40
SGOT	122.56	56.70	26.35	80.60	114.70	151.90	457.25
Tryglicerides	124.70	65.15	33.00	84.25	108.00	151.00	598.00
Platelets	257.02	98.33	62.00	188.50	251.00	318.00	721.00
Prothrombin	10.73	1.02	9.00	10.00	10.60	11.10	18.00

Część cech z badań krwi jest mocno skośna, czyli ma nierówny rozkład. Najlepiej widać to przy bilirubinie i Alk_Phos: u większości pacjentów wartości są niskie, ale kilku ma je skrajnie wysokie. Przy bilirubinie mediana to tylko 1,4, a maksimum aż 28 - czyli garstka ekstremalnych przypadków zawyża średnią. Tak samo zachowują się cholesterol, miedź i SGOT. Albumina jest odwrotna: prawie wszystkie wyniki są podobne (około 3,5), a wyjątki to wartości bardzo niskie. Dlatego braki uzupełniam medianą, a nie średnią - mediany nie psują pojedyncze skrajne wyniki.

Cechy kateryczne.

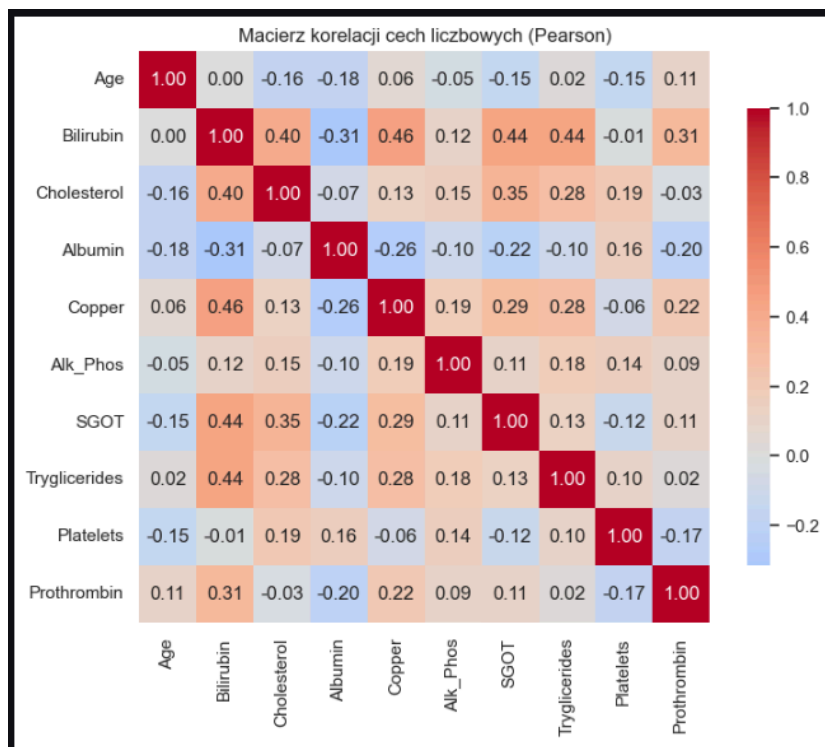
	cecha	wartość	liczność
0	Drug	D-penicillamine	158
1	Drug	Placebo	154
2	Drug	nan	105
3	Drug	NaN	1
4	Sex	F	374
5	Sex	M	44
6	Ascites	N	288
7	Ascites	nan	105
8	Ascites	Y	24
9	Ascites	NaN	1
10	Hepatomegaly	Y	160
11	Hepatomegaly	N	152
12	Hepatomegaly	nan	105
13	Hepatomegaly	NaN	1
14	Spiders	N	222
15	Spiders	nan	105
16	Spiders	Y	90
17	Spiders	NaN	1
18	Edema	N	354
19	Edema	S	44
20	Edema	Y	20
21	Stage	3.0	155
22	Stage	4.0	144
23	Stage	2.0	92
24	Stage	1.0	21
25	Stage	nan	6

Braki danych.

	n_missing	pct_missing
Tryglicerides	136	32.5
Cholesterol	134	32.1
Copper	108	25.8
Alk_Phos	106	25.4
SGOT	106	25.4
Drug	105	25.1
Spiders	105	25.1
Ascites	105	25.1
Hepatomegaly	105	25.1
Platelets	11	2.6
Stage	6	1.4
Prothrombin	2	0.5

Braki nie są przypadkowe. Sześć kolumn - Drug, Ascites, Hepatomegaly, Spiders, Alk_Phos i SGOT - ma po około 105-106 braków, czyli mniej więcej 25% zbioru. To pokazuje, że chodzi o tę samą grupę pacjentów: około 106 osób, które nie brały udziału we właściwym badaniu klinicznym i dla których nie zebrano pełnego zestawu danych. Trójglicerydy, cholesterol i miedź mają braków trochę więcej (26-32%), bo doszły pojedyncze brakujące pomiary. Reszta kolumn jest prawie kompletna. Cała kolumna Drug jest pusta dla tych pacjentów, dlatego brak potraktowałem jako osobną kategorię, zamiast wyrzucać wiersze i tracić ćwierć danych.

Korelacje.



Cechy liczbowe są ze sobą słabo lub umiarkowanie powiązane, więc nie ma silnej współliniowości, która psułaby modele - najwyższa korelacja to tylko 0,46. Najbardziej z innymi powiązana jest bilirubina: rośnie razem z miedzią (0,46), SGOT i trójglicerydami (po 0,44) oraz cholesterolem (0,40), a maleje, gdy rośnie albumina (-0,31). To ma sens medyczny, bo wszystkie te wartości to markery pracy wątroby. Powiązania są jednak na tyle umiarkowane, że nie usuwałem żadnej cechy.

5. Przygotowanie danych

5.1. Usunięcie wycieku i identyfikatora

Zgodnie z uzasadnieniem z sekcji 2, przed dalszym przetwarzaniem usuwam kolumny `N_Days` (wyciek targetu) oraz `ID` (identyfikator). Pozostaje 17 cech zgodnych z treścią zadania oraz kolumna docelowa `Status`.

5.2. Podział danych

Do oceny stosuję podział stratyfikowany - `train_test_split(..., stratify=Status)` dla zbioru holdout oraz `StratifiedKFold(n_splits=5)` przy strojeniu hiperparametrów. Stratyfikacja jest tu konieczna ze względu na rzadką klasę `CL`.

	train_%	holdout_%
Status		
C	55.4	56.0
CL	6.0	6.0
D	38.6	38.1

5.3. Postępowanie z brakami danych

Braki uzupełniam imputacją, świadomie nie usuwając wierszy - chcę zachować jak najwięcej informacji, a usunięcie ~106 pacjentów z brakami oznaczałoby utratę ćwierci zbioru. Przyjmuję:

- cechy liczbowe - imputacja medianą (odporna na wartości odstające, których w danych laboratoryjnych jest sporo),
- cechy kateryczne - uzupełnienie wartości `Missing` jako osobną kategorią, co zachowuje informację o samym fakcie braku (istotne np. dla `Drug`, gdzie brak dotyczy konkretnej, nielosowej podgrupy pacjentów).

5.4. Porównanie metod przetwarzania

Zgodnie z wymaganiem porównuję wynik bez przetwarzania (baseline) z wynikami po zastosowaniu co najmniej dwóch metod, każdej osobno. Świadomie nie wybieram do tego porównania standaryzacji ani normalizacji - jak zaznaczyłem w sekcji 3.2, nie zmieniają one wyników gaussowskiego Bayesa ani drzewa decyzyjnego, więc dla tych dwóch klasyfikatorów takie porównanie dałoby identyczne liczby. Zamiast tego porównuję metody, które realnie wpływają na te modele:

Wariant	Opis	Cel
Baseline	imputacja + kodowanie one-hot cech kategorycznych	punkt odniesienia
Selekcja cech	dodatkowo SelectKBest z miarą informacji wzajemnej (odpowiednik przyrostu informacji)	redukcja szumu, mniej cech
PCA	standaryzacja + PCA (redukcja do składowych głównych)	redukcja wymiaru, dekorelacja

Wszystkie kroki przetwarzania osadziłem w Pipeline scikit-learn, tak aby ich dopasowanie (fit) odbywało się wyłącznie na zbiorze treningowym - również wewnątrz walidacji krzyżowej. Zapobiega to wyciekowi statystyk ze zbioru walidacyjnego do procesu uczenia.

Do selekcji cech użyłem SelectKBest z $k=12$ - zostawiam 12 najbardziej informatywnych cech z 32, które powstają po zakodowaniu zmiennych kategorycznych (one-hot). PCA ustawiłem na 8 składowych głównych, czyli redukuję wymiar z 32 do 8. Obie wartości dobrałem tak, żeby wyraźnie zmniejszyć liczbę cech i sprawdzić, czy taka redukcja pomaga modelom.

Tabela porównawcza (baseline vs selekcja vs PCA, dla obu klasyfikatorów):

	model	preprocessing	f1_macro_cv	f1_macro_std	accuracy_cv	f1_weighted_cv
0	DecisionTree	baseline	0.5070	0.0650	0.6408	0.6415
1	DecisionTree	feature_select	0.4800	0.0773	0.6350	0.6280
2	DecisionTree	pca	0.4270	0.0816	0.5959	0.5888
3	GaussianNB	feature_select	0.5373	0.0754	0.6916	0.6825
4	GaussianNB	baseline	0.5127	0.0347	0.6377	0.6558
5	GaussianNB	pca	0.4689	0.0271	0.7035	0.6744

Z porównania widać dwie rzeczy. Po pierwsze, PCA wypadło najgorzej dla obu modeli pod względem F1-macro (drzewo 0,43, NB 0,47), mimo że dla NB dało najwyższą dokładność (0,73). To dobrze pokazuje pułapkę PCA przy niezbalansowanych klasach: poprawia ogólną dokładność, ale spycha model w stronę najczęstszej klasy, przez co gorzej radzi sobie z rzadkimi - a to właśnie mierzy macro-F1. Po drugie, dla drzewa najlepszy okazał się baseline (0,51), a selekcja cech delikatnie pogorszyła wynik - drzewo samo wybiera najważniejsze cechy przy podziałach, więc dokładanie SelectKBest niewiele wnosi. Dla naiwnego Bayesa było odwrotnie: selekcja cech pomogła (0,54

wobec 0,51 dla baseline), bo ten model traktuje wszystkie cechy równo i jest czuły na te mniej istotne, więc ich odrzucenie poprawiło wynik.

6. Klasyfikacja

6.1. Strojenie hiperparametrów

Do doboru hiperparametrów używam `GridSearchCV` z 5-krotną walidacją stratyfikowaną. Jako kryterium optymalizacji ustawiam `f1_macro`, a nie dokładność - z powodu niezbalansowania klas zależy mi na modelu radzącym sobie ze wszystkimi klasami, a nie tylko z większością.

Naiwny klasyfikator Bayesa. Strojony parametr to `var_smoothing` (wygładzanie wariancji), testowany dla co najmniej trzech wartości.

	<code>var_smoothing</code>	<code>f1_macro_cv</code>	<code>std</code>	<code>f1_macro_train</code>	<code>rank</code>
0	1.000000e-09	0.512675	0.034728	0.608068	1
1	1.000000e-07	0.506395	0.071849	0.547613	2
2	1.000000e-05	0.462383	0.064834	0.515202	3
3	1.000000e-03	0.374855	0.038936	0.418275	4

Najlepsza konfiguracja: `var_smoothing=1e-9`

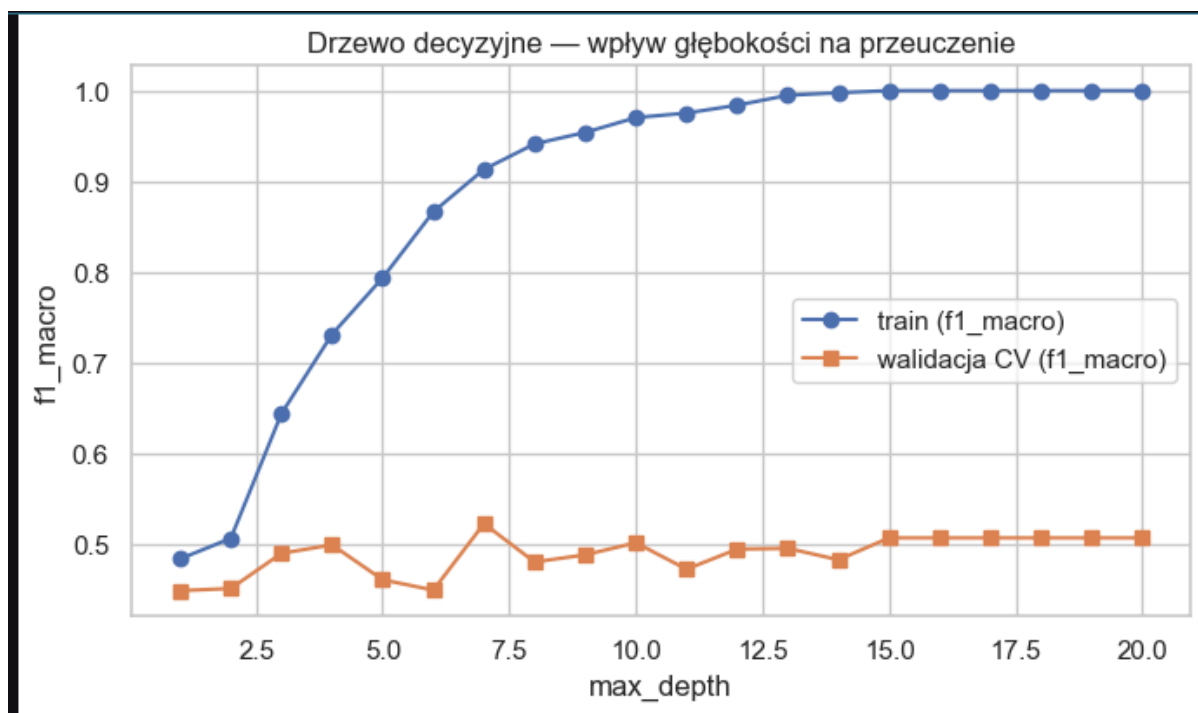
Drzewo decyzyjne. Przeszukuję siatkę po `criterion` (gini / entropy), `max_depth` oraz `min_samples_leaf`.

	<code>criterion</code>	<code>max_depth</code>	<code>min_samples_leaf</code>	<code>f1_macro_cv</code>	<code>std</code>	<code>f1_macro_train</code>	<code>rank</code>
0	entropy	5	5	0.556046	0.093650	0.775883	1
1	entropy	10	5	0.544745	0.082581	0.812671	2
2	entropy	None	5	0.544745	0.082581	0.812671	2
3	entropy	5	1	0.534857	0.086943	0.831657	4
4	entropy	None	1	0.532656	0.085916	1.000000	5
5	entropy	10	1	0.530529	0.080115	0.994502	6
6	entropy	5	10	0.507403	0.058503	0.650120	7
7	gini	None	1	0.506964	0.064986	1.000000	8
8	entropy	None	10	0.503059	0.046515	0.654286	9
9	entropy	10	10	0.503059	0.046515	0.654286	9

Najlepsza konfiguracja: `criterion=entropy`, `max_depth=5`, `min_samples_leaf=5`

Strojenie różnie wpłynęło na oba modele. Naiwny Bayes ma niewiele do strojenia - jedyny parametr to `var_smoothing`, a najlepiej wypadła najmniejsza wartość. Ten model nie przeucza się mocno: wynik na treningu (0,61) jest tylko trochę wyższy niż na walidacji (0,51). Zupełnie inaczej zachowuje się drzewo. Bez ograniczenia głębokości uczy się danych treningowych na pamięć - na wykresie

widać, że przy rosnącej głębokości wynik na treningu dochodzi do 1,0, a na walidacji stoi w miejscu w okolicach 0,5. To podręcznikowy przykład przeuczenia. Dlatego najlepsze okazało się płytkie drzewo (głębokość 5), które nie dopasowuje się idealnie do treningu, ale lepiej radzi sobie z nowymi danymi.



6.2. Zadanie bonusowe - algorytmy zaawansowane

W ramach zadania bonusowego skupiłem się na maszynie wektorów nośnych (SVM). Dla porównania uruchomiłem też Random Forest, którego wynik znajduje się w tabeli zbiorczej w sekcji 7.

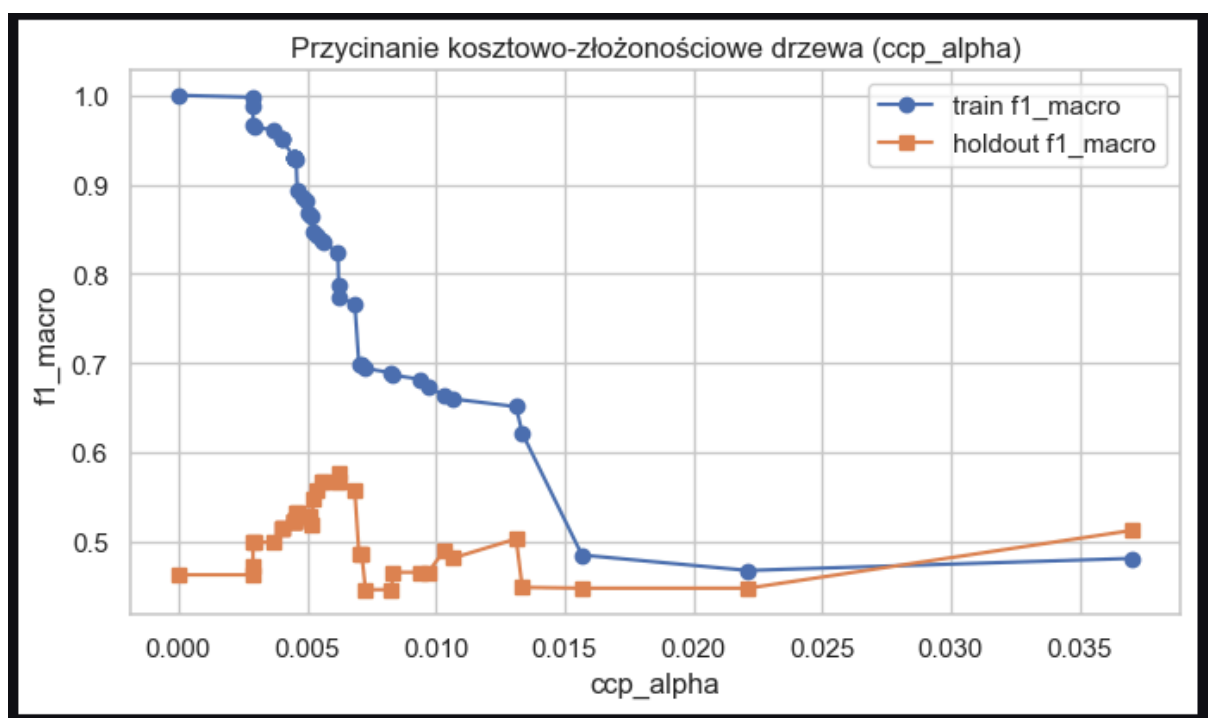
	C	kernel	gamma	f1_macro_cv	rank
0	10.0	linear	auto	0.538570	1
1	10.0	linear	scale	0.538570	1
2	1.0	linear	auto	0.530906	3
3	1.0	linear	scale	0.530906	3
4	10.0	rbf	scale	0.509475	5
5	1.0	rbf	auto	0.500113	6
6	0.1	linear	auto	0.495544	7
7	0.1	linear	scale	0.495544	7

Przy SVM zastosowałem wariant ze standaryzacją cech (poprzez pipeline / PCA), ponieważ - w odróżnieniu od drzewa - SVM jest wrażliwy na skalę cech: bez ujednolicenia skali cechy o dużych wartościach liczbowych zdominowałyby wyznaczanie marginesu. Drzewo takiego zabiegu nie wymaga, bo dzieli każdą cechę niezależnie według progów, na które przeskalowanie nie ma wpływu.

SVM nie pobił dwóch podstawowych modeli tak wyraźnie, jak można by oczekiwać. W walidacji krzyżowej osiągnął F1-macro około 0,54, a na zbiorze testowym wypadł w środku stawki - lepiej od nieprzyciętego drzewa, ale gorzej od drzewa przyciętego. Najlepiej widać za to znaczenie skalowania: bez ujednolicenia skali cech wynik SVM był wyraźnie słabszy (F1-macro około 0,29), a po standaryzacji wzrósł do poziomu pozostałych modeli. To potwierdza, że SVM jest czuły na skalę cech, a drzewo nie.

6.3. Zadanie bonusowe - łagodzenie przeuczenia

Jako podejście do łagodzenia przeuczenia wybrałem **przycinanie kosztowo-złożonościowe** (`ccp_alpha`) dla drzewa decyzyjnego. Dla rosnących wartości `ccp_alpha` drzewo jest coraz mocniej przycinane, co zmniejsza jego złożoność i powinno zredukować różnicę między dokładnością treningową a walidacyjną.



	konfiguracja	train_f1_macro	holdout_f1_macro	różnica_train-holdout
0	drzewo pełne (ccp_alpha≈0)	1.0000	0.4628	0.5372
1	drzewo przycięte (ccp_alpha=0.00621)	0.7878	0.5767	0.2111

Przycinanie zadziałało dokładnie tak, jak powinno. Pełne drzewo miało na treningu idealny wynik 1,0, ale na teście tylko 0,46 - różnica 0,54, czyli klasyczne przeuczenie. Po przycięciu (`ccp_alpha = 0,0062`) wynik na treningu spadł do 0,79, za to na teście wzrósł do 0,58, a różnica między nimi zmalała z 0,54 do 0,21. Czyli przycinanie nie tylko zmniejszyło przeuczenie ponad dwukrotnie, ale przy okazji poprawiło wynik na danych, których model nie widział. To było najskuteczniejsze podejście w całym zadaniu.

7. Ewaluacja

Dla najlepszych konfiguracji każdego klasyfikatora zebrałem komplet metryk na zbiorze walidacyjnym. Ze względu na trzy niezbalansowane klasy raportuję zarówno uśrednienia macro, jak i weighted, a także wartości per-klasa.

### GaussianNB (strojony)				
	precision	recall	f1-score	support
C	0.712	0.787	0.747	47.000
CL	0.100	0.200	0.133	5.000
D	0.682	0.469	0.556	32.000
accuracy	0.631	0.631	0.631	0.631
macro avg	0.498	0.485	0.479	84.000
weighted avg	0.664	0.631	0.638	84.000

### DecisionTree (strojony)				
	precision	recall	f1-score	support
C	0.672	0.872	0.759	47.000
CL	0.111	0.200	0.143	5.000
D	0.714	0.312	0.435	32.000
accuracy	0.619	0.619	0.619	0.619
macro avg	0.499	0.462	0.446	84.000
weighted avg	0.655	0.619	0.599	84.000

Tabela zbiorcza:

	model	accuracy	precision_macro	recall_macro	f1_macro	precision_weighted	recall_weighted	f1_weighted	f1_C	f1_CL	f1_D
0	DecisionTree (ccp przycięte)	0.7381	0.5779	0.5758	0.5767	0.7375	0.7381	0.7377	0.8000	0.2000	0.7302
1	RandomForest (strojony)	0.7381	0.4940	0.5095	0.4987	0.6954	0.7381	0.7121	0.7961	0.0000	0.7000
2	SVM (skalowany, strojony)	0.7024	0.4846	0.4883	0.4842	0.6795	0.7024	0.6877	0.7525	0.0000	0.7000
3	GaussianNB (strojony)	0.6310	0.4978	0.4853	0.4788	0.6638	0.6310	0.6378	0.7475	0.1333	0.5556
4	DecisionTree (strojony)	0.6190	0.4992	0.4616	0.4456	0.6548	0.6190	0.5990	0.7593	0.1429	0.4348

Macierze pomyłek:

GaussianNB (strojony)				DecisionTree (strojony)				DecisionTree (ccp przycięte)				RandomForest (strojony)				SVM (skalowany, strojony)			
rzeczywistość	C	CL	D	rzeczywistość	C	CL	D	rzeczywistość	C	CL	D	rzeczywistość	C	CL	D	rzeczywistość	C	CL	D
	37	4	6		41	2	4		38	1	8		41	0	6		38	2	7
	3	1	1		4	1	0		4	1	0		4	0	1		5	0	0
predycja	C	CL	D	predycja	C	CL	D	predycja	C	CL	D	predycja	C	CL	D	predycja	C	CL	D
	12	5	15		16	6	10		6	3	23		11	0	21		11	0	21

Najważniejsza obserwacja to duża różnica między dokładnością a macro-F1 u wszystkich modeli (np. drzewo przycięte: dokładność 0,74, ale macro-F1 0,58). Bierze się to wprost z niezbalansowania. Modele dobrze radzą sobie z dwiema licznymi klasami (C i D), więc dokładność jest wysoka, ale słabo z rzadką CL, a macro-F1 liczy każdą klasę równo, więc od razu to widać.

Macierze pomyłek pokazują to samo od strony klas. Klasa CL prawie nigdy nie jest rozpoznawana poprawnie - modele trafiają najwyżej 1 z 5 przypadków, a resztę wrzucają do C albo D. Powód jest prosty: CL ma w całym zbiorze tylko 25 przypadków, a w zbiorze testowym około pięciu, więc model nie ma się z czego nauczyć, jak ta klasa wygląda.

Porównując dwa wymagane modele, naiwny Bayes i drzewo wyszły bardzo blisko siebie, z lekką przewagą Bayesa w macro-F1 (0,48 wobec 0,45), głównie dlatego, że trochę lepiej rozpoznaje klasę D. Założenie Bayesa o niezależności cech, mimo że nie jest tu w pełni prawdziwe, nie zaszkodziło - model zadziałał porównywalnie z drzewem.

Najlepszą konfiguracją okazało się przycięte drzewo decyzyjne, które miało i najwyższą dokładność (0,74), i najwyższy macro-F1 (0,58), wyprzedzając Random Forest oraz SVM. Pokazuje to, że na tych danych większe znaczenie miało ograniczenie przeuczenia niż sięganie po bardziej złożony algorytm.

8. Wnioski

Najskuteczniejszym modelem do prognozowania przeżywalności okazało się przycięte drzewo decyzyjne - po ograniczeniu przeuczenia osiągnęło najlepszy wynik zarówno w dokładności, jak i w macro-F1, bijąc bardziej złożone Random Forest i SVM. Pokazuje to, że dobranie odpowiedniej regularyzacji potrafi dać więcej niż sam wybór mocniejszego algorytmu.

Głównym ograniczeniem wszystkich modeli jest słabe rozpoznawanie klasy mniejszościowej CL. Wynika to wprost z niezbalansowania zbioru - przy zaledwie 25 przypadkach na 418 model nie ma się z czego nauczyć tej klasy, dlatego prawie zawsze myli ją z dwiema licznymi.

Świadomie usunąłem kolumnę N_Days, bo jest to wyciek informacji (zawiera czas do zgonu, czyli pośrednio odpowiedź). Skutkiem jest to, że moje wyniki są niższe, niż gdybym ją zostawił, ale za to są uczciwe i mówią o rzeczywistej zdolności modelu do przewidywania na podstawie danych klinicznych.

W przyszłości wynik dla rzadkiej klasy można by poprawić technikami sztucznego balansowania klas (np. SMOTE) oraz dokładniejszym traktowaniem wartości odstających.

9. Użyte narzędzia i biblioteki

Implementację wykonałem w Pythonie, w środowisku Jupyter Notebook. Wykorzystane biblioteki:

- **scikit-learn** - modele klasyfikacji (GaussianNB, DecisionTreeClassifier, RandomForestClassifier, SVC), przetwarzanie danych (SimpleImputer, OneHotEncoder, StandardScaler, PCA, SelectKBest), strojenie (GridSearchCV, StratifiedKFold) oraz metryki oceny.
- **pandas** i **numpy** - wczytanie i obróbka danych tabelarycznych.
- **matplotlib** i **seaborn** - wizualizacje (rozkłady cech, macierze pomyłek, krzywe).
- **ucimlrepo** - pobranie zbioru bezpośrednio z repozytorium UCI.

10. Materiały Źródłowe

1. Dickson, E., Grambsch, P., Fleming, T., Fisher, L., & Langworthy, A. (1989). *Cirrhosis Patient Survival Prediction* [Dataset]. UCI Machine Learning Repository.
<https://doi.org/10.24432/C5R02G>
2. Dokumentacja biblioteki scikit-learn, <https://scikit-learn.org/stable/>
3. Materiały wykładowe z przedmiotu Sztuczna Inteligencja i Inżynieria Wiedzy, Politechnika Wrocławska, 2026.
4. Gemini Pro